

Aflevering P4

Mikkel Moth Billing

Denne aflevering indeholder tre opgaver, der kan løses uafhængigt.

Første opgave går ud på at plotte og animere en Sangaku-inspireret tegning.

Den anden opgave er endnu en opgave inspireret af de opgaver du lavede i kapitel 13 i kurset Calculus i sidste semester. Du ser nok at det er meget nemmere at løse denne opgave numerisk end det ville have været at løse den analytisk som du ville skulle have gjort i Calculus.

Den tredje opgave er en vektoriel anden ordens differentialligning, der skal løses og animeres.

Spørgsmål 1

Gengiv den feedback du fik i forbindelse med Aflevering P3 og svar på evt. spørgsmål instruktoren har stillet.

Til P2 fik jeg at vide, at jeg kunne dele min figur funktion op i mindre funktioner, så det har jeg prøvet.

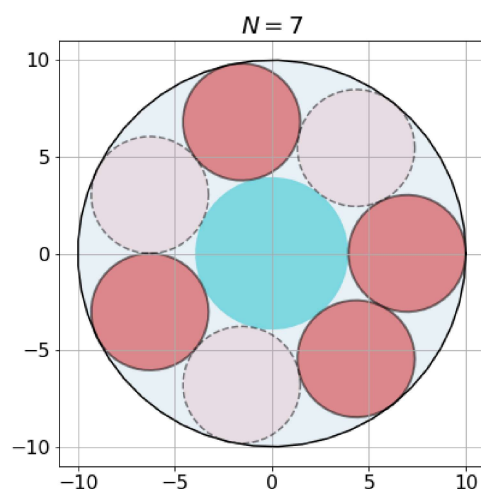
Sangaku-opgaven

Se på Sangaku-tegningerne i opgave P3. I denne opgave skal du arbejde med en lignende tegning for $N = 7$. Benyt:

$$b = 10$$

$$a = 3.94812012$$

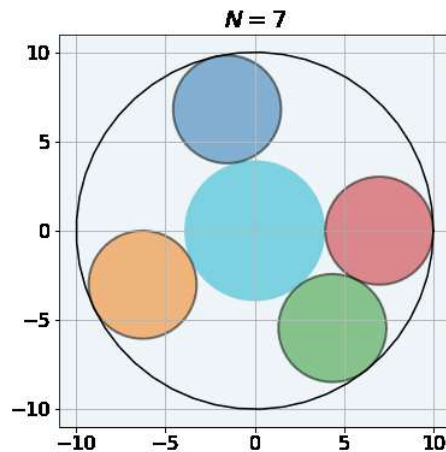
men plot ikke alle 7 små cirkler (dem med radius c), plot kun hver anden af dem så du har i alt 4 stk på de oprindelige positioner. Dermed vil der præcis være to af dem, der rører hinanden.



Du skal planlægge løsningen af opgaven, så du kan modificere de små cirkler. I første spørgsmål skal de have nye farver og i andet spørgsmål skal de flyttes rundt.

Spørgsmål 2

Lav en animation, hvor du skifter farverne af cirklerne. Du kan gøre det efter et system, du selv bestemmer, eller du kan gøre det tilfældigt. Forklar dit valg i en kommentarsætning og omtal det i videoen.



Jeg har brugt `random.random()` til at vælge en tilfældig RGB farve, som hver cirkel skifter til i hver frame.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fmin
from matplotlib import animation
import random

plt.rc("xtick", labelsizes=10)
plt.rc("ytick", labelsizes=10)
plt.rc("axes", titlesizes=16)
plt.rc("font", size=16)
plt.rc("axes", grid='on')
plt.rc('animation', html='jshtml')
```

```
In [2]: fig, ax = plt.subplots()
plt.close()

t = np.linspace(0, 2*np.pi, 100)

Rot_mat = lambda v: np.array([[np.cos(np.pi/180*v), -np.sin(np.pi/180*v)],
                               [np.sin(np.pi/180*v), np.cos(np.pi/180*v)]])

def lav_c_cirkler(a,b,N):
    c = (b-a)/2
    cm = np.array([a+c, 0])
    c_fig_liste = []
    c_cm_liste = []
    for i in range(N):
        vinkel = 360/N
        if i%2 == 0:
            cfig = ax.fill(c*np.cos(t)+cm[0], c*np.sin(t)+cm[1], color="tab:red", alpha=0.5)
            c_fig_liste.append(cfig[0])
            c_cm_liste.append(cm)
            cm = np.dot(Rot_mat(vinkel), cm)
    return c_fig_liste, c_cm_liste

def lav_figur(a,b,N):
    ax.fill(a*np.cos(t), a*np.sin(t), color="tab:cyan", alpha=0.5)
    ax.plot(b*np.cos(t), b*np.sin(t), "k")

    c_fig_liste, c_cm_liste = lav_c_cirkler(a,b,N)

    ax.set_title(f"N={N}")
    ax.set_aspect('equal')
```

```

ax.xaxis.set_major_locator(plt.MultipleLocator(5))
ax.yaxis.set_major_locator(plt.MultipleLocator(5))
return c_fig_liste, c_cm_liste

b = 10
a = 3.94812012

c_fig_liste = lav_figur(a,b,7)[0]

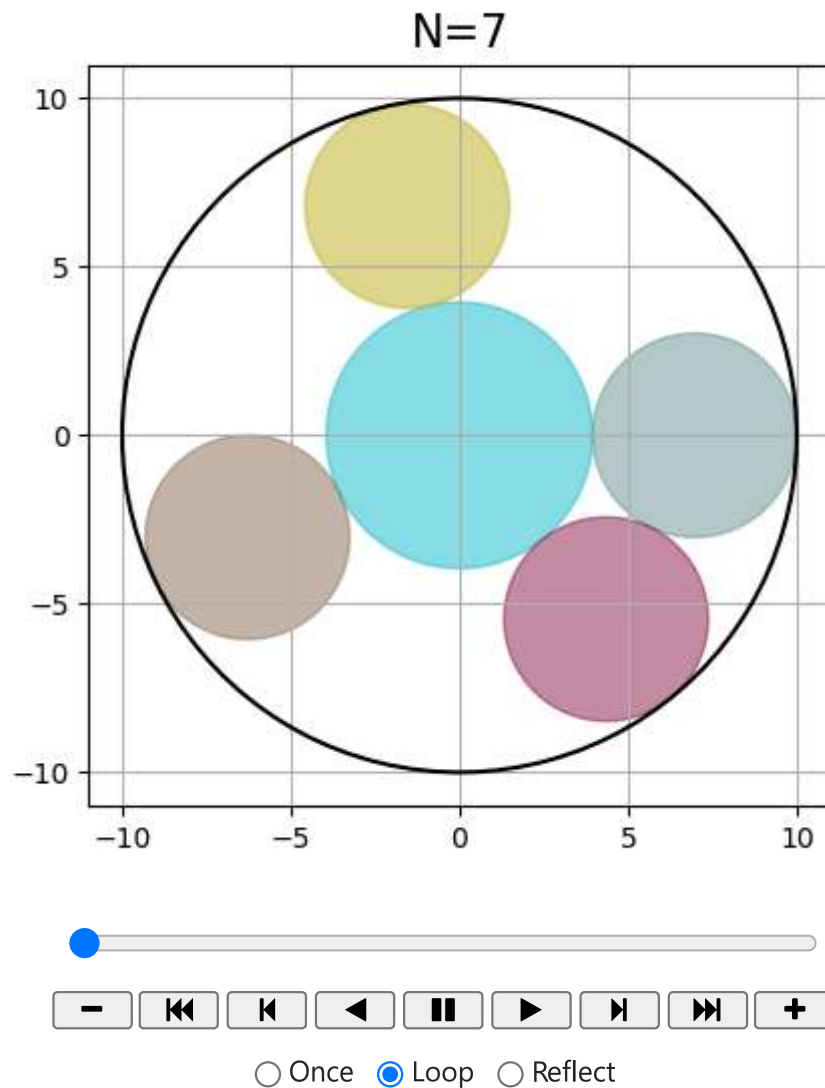
def update(i):
    for c_fig in c_fig_liste:
        rgba = (random.random(),random.random(),random.random(), random.random())
        c_fig.set_color(rgba)
    return c_fig_liste

anim = animation.FuncAnimation(fig,
                                update,
                                frames=10,
                                interval=400,
                                blit=True)

anim

```

Out[2]:



Spørgsmål 3

Skriv nu en update-funktion, der kan håndtere at modtage et argument, `i`. Den skal kun se på den del af `i`, der er tilbage som rest efter division med 40. Skriv fx:

```

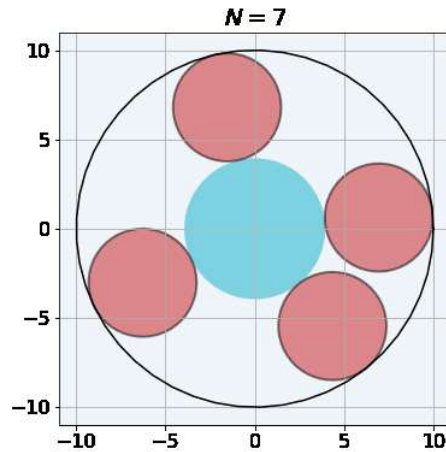
i = 52
rest = i % 40
rest

```

12

Hvis $0 \leq \text{rest} < 10$ skal update-funktionen flytte den første røde skive en vinkel $360^\circ/70$ rundt i rummet mellem de blå skiver. Hvis $10 \leq \text{rest} < 20$ skal den i stedet flytte den anden røde skive tilsvarende, osv.

Benyt din update-funktion til at lave en animation af 70 frames.



Hint 1: Hav en liste med de fire røde skivers centre, som du holder styr på. Hver gang en skive skal flyttes:

- regner du det nye centrum ud og gemmer i listen
- regner du x- og y-koordinaterne for cirkels rand ud
- opdaterer du figur-elementet. Til TØ gjorde vi dette for et kvadrat v.h.j.a `.set_xy()` -metoden.

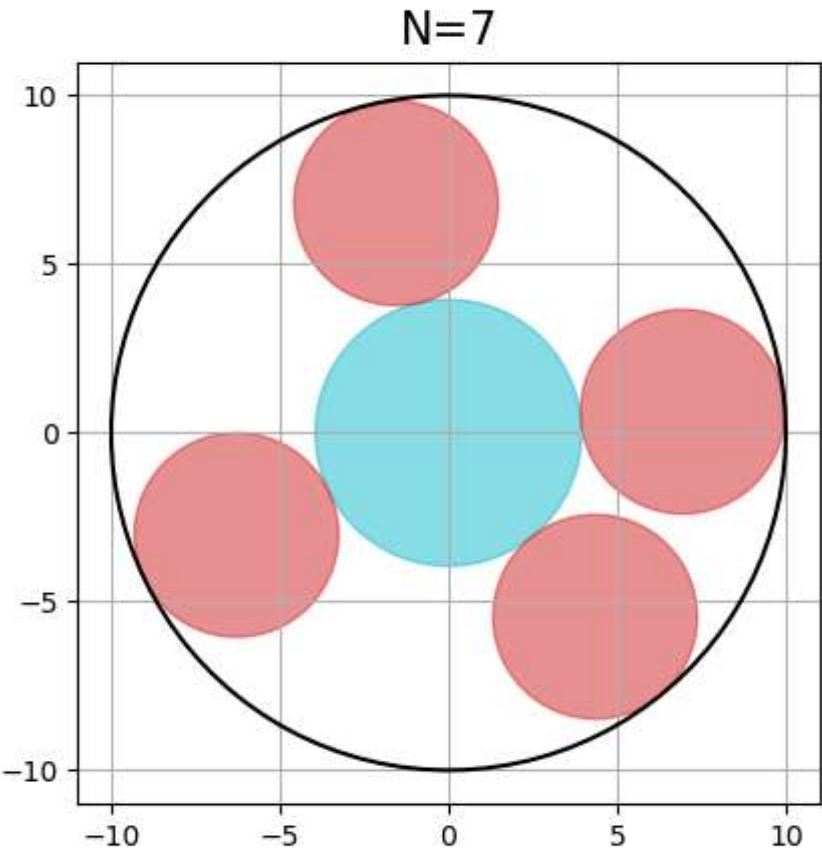
Hint 2: Tilføj dette til argumenterne i kaldet af `animation.FuncAnimation`:

```
init_func=lambda : []
```

så slipper du for at den kalder din update-funktion flere gange med `i=0` til at begynde med.

[illegible]

Out[6]:



-

⏮

⏪

◀

⏸

▶

⏩

⏭

+

☐ Once

☒ Loop

☐ Reflect